.UBAfiuba
FACULTAD DE INGENIERÍA

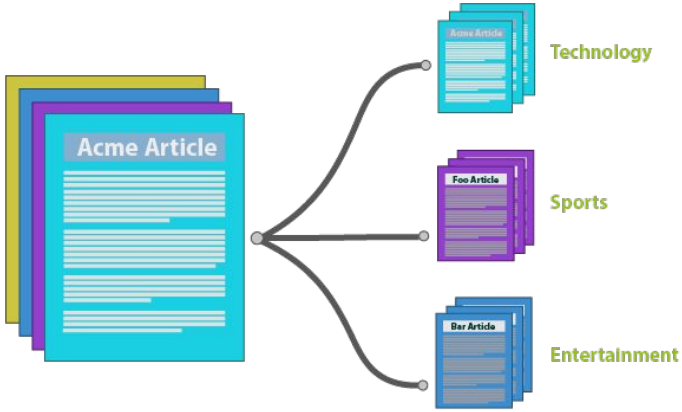
NLP

CNN para NLP

Docente:
MSc. Fernando Silva

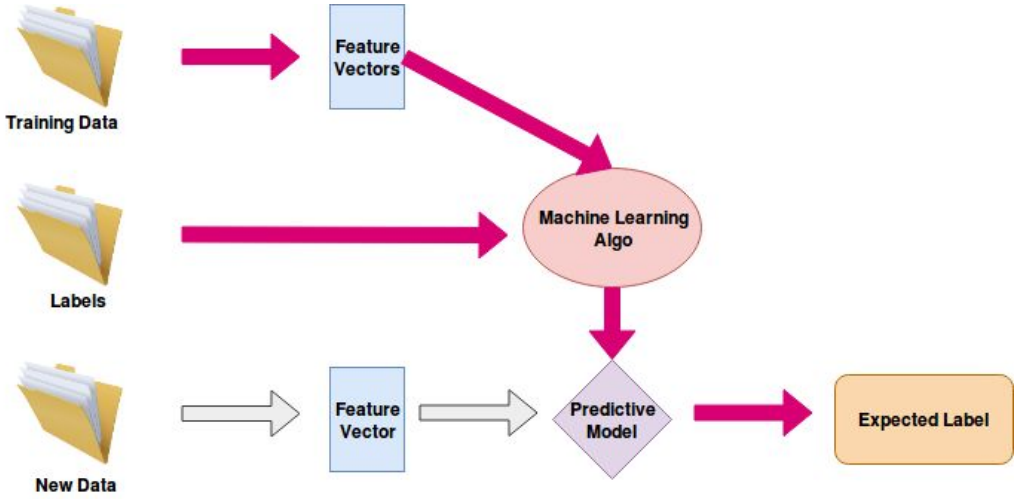
email: fernandosilva.clases@gmail.com

Modelo de clasificación de texto



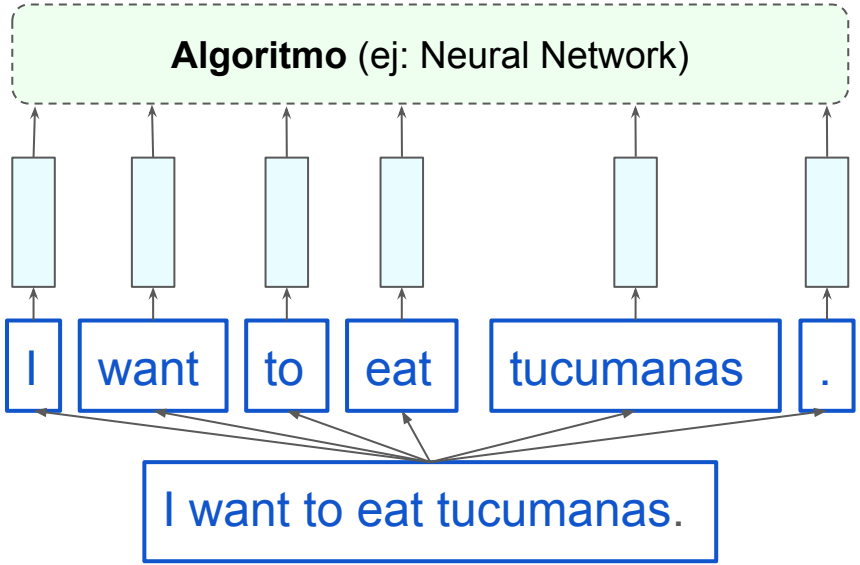
Preprocesamiento:

- Segmentar, tokenizar, etc…



Representación de Secuencias

Para realizar **tareas de NLP con Deep Learning**, necesitamos representaciones vectoriales que capten de manera más precisa el significado y las relaciones entre palabras; es decir, **necesitamos embeddings.**

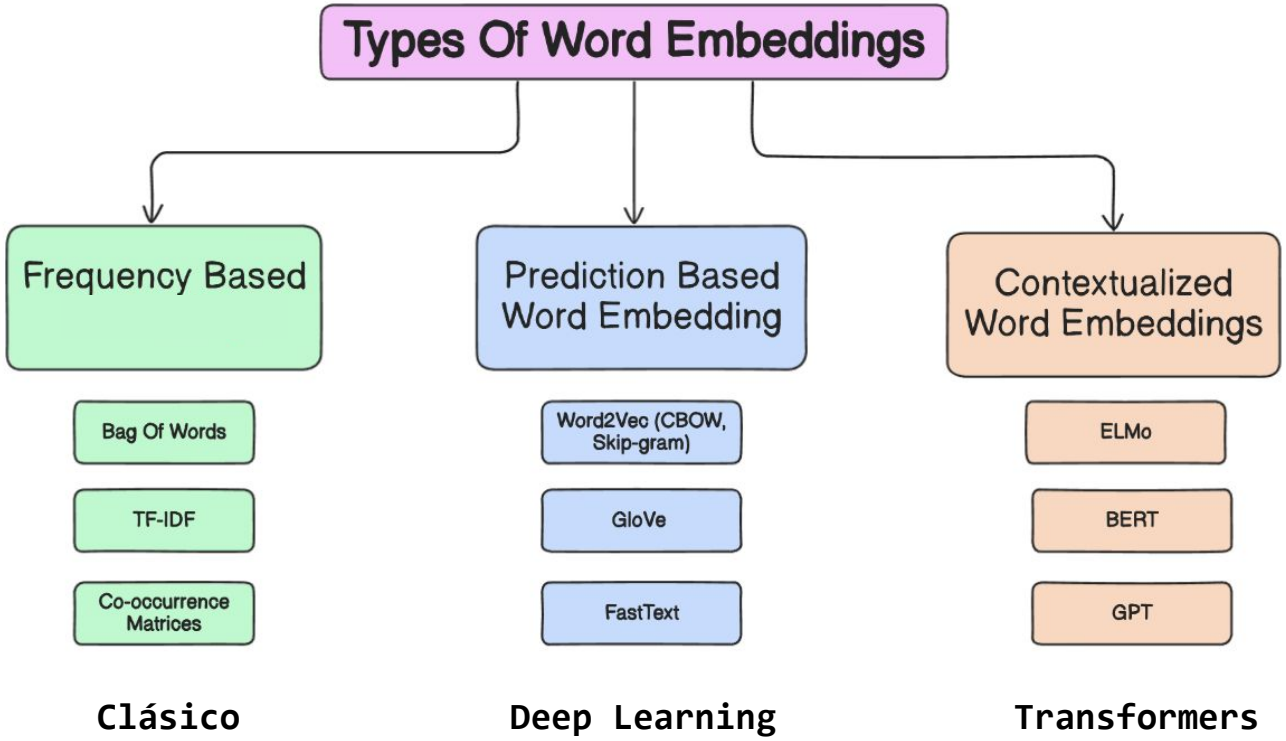


Word representation *(usualmente embeddings)*

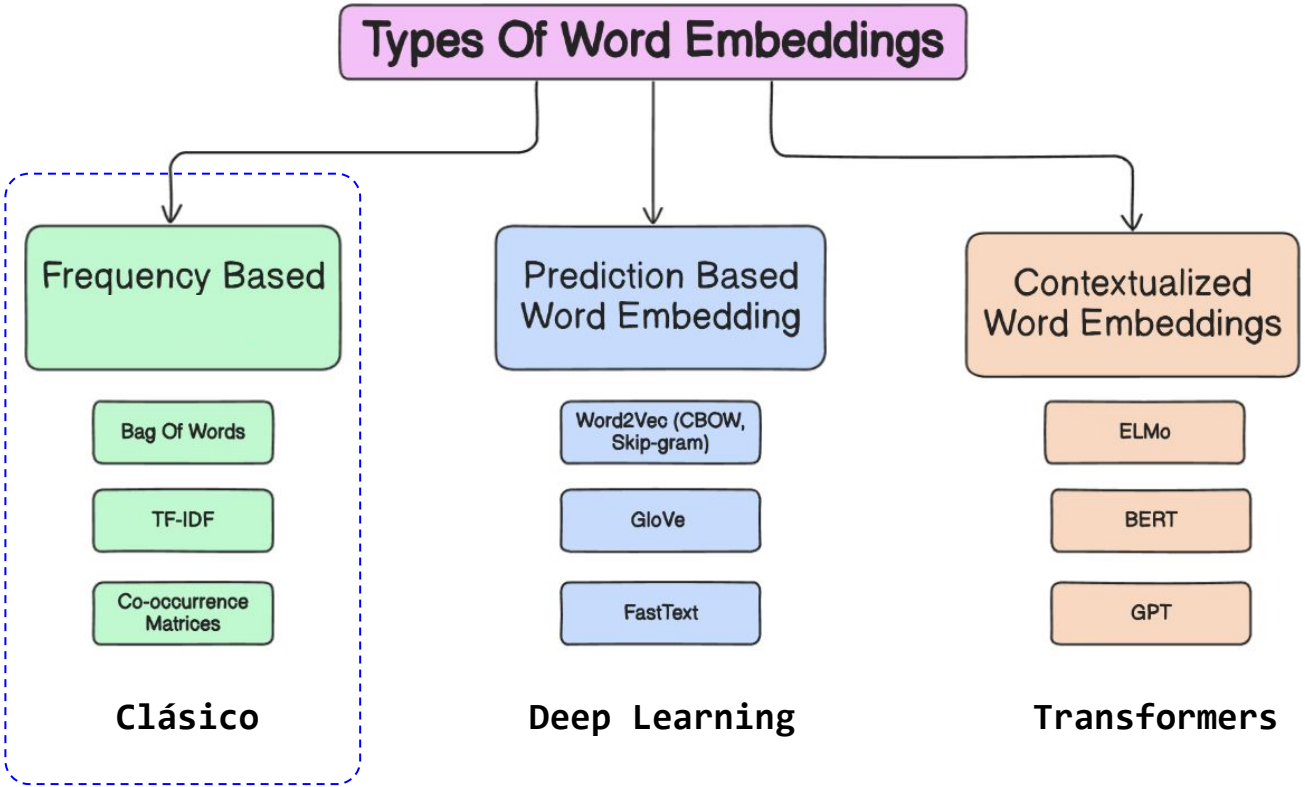
Secuencia de ***tokens***

Texto (***input***)

Representación de Secuencias



Representación de Secuencias



One-Hot Encoding

Empecemos con algo básico,
representación vectorial **One Hot Vector**

Hello, I'm Ironman. I have Friday AI

One Hot Encoding

AI	1	0	0	0	0	0	0
Ironman	0	1	0	0	0	0	0
Friday	0	0	1	0	0	0	0
have	0	0	0	1	0	0	0
Hello	0	0	0	0	1	0	0
I	0	0	0	0	0	1	0
I'm	0	0	0	0	0	0	1

Alta dimensionalidad. No generalizan. Ignoran el contexto

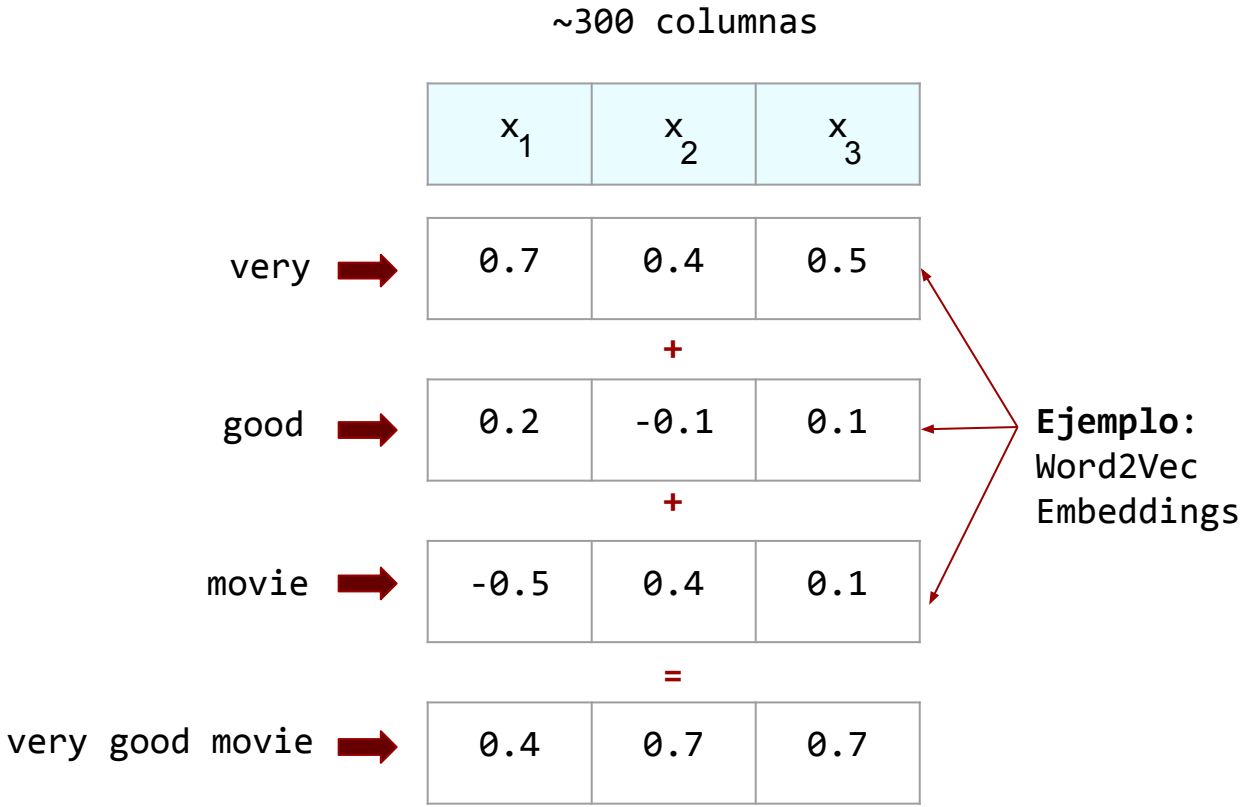
Representación por One-Hot Encoding

https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/may_2026/Clase%205/C%C3%B3digo/BOW_y_MLP.ipynb

~100K columnas

	good	movie	very	a	did	like
very →	0	0	1	0	0	0
+						
good →	1	0	0	0	0	0
+						
movie →	0	1	0	0	0	0
=						
very good movie →	1	1	1	0	0	0

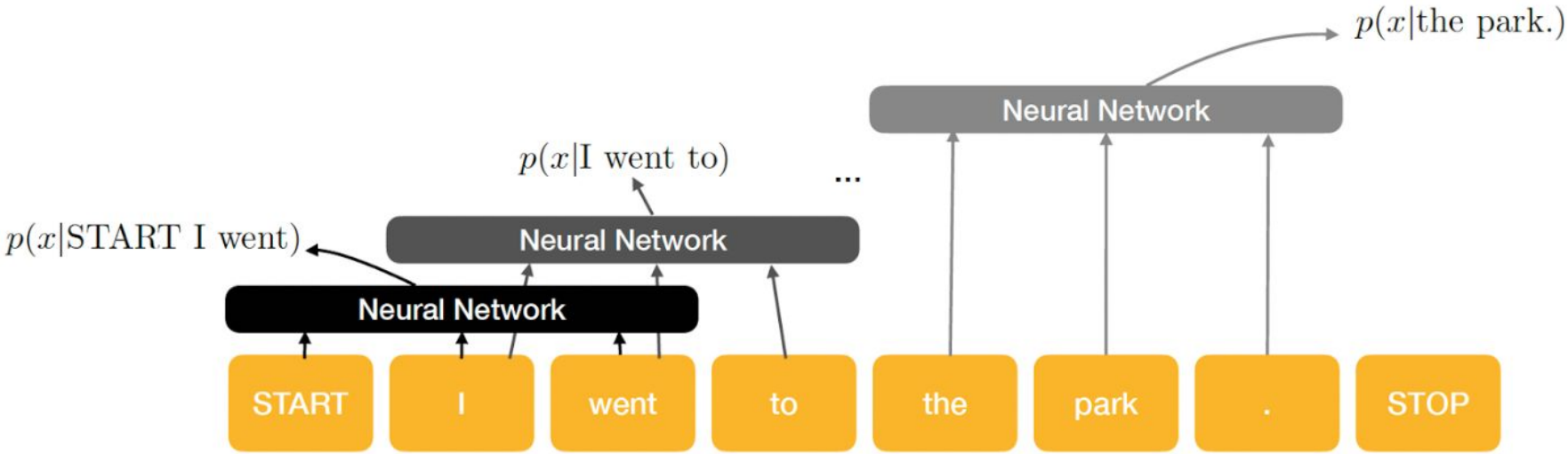
Representación por Embedding



CNN para Secuencias

Las redes neuronales que vimos hasta ahora reciben como entrada vectores de tamaño fijo...

Idea 1: Sliding window de tamaño N (CNN de 1D)



CNN para Secuencias

- En vez de pixeles de imágenes, la entrada en tareas de NLP son oraciones o documentos representados como una matriz.

- De esta matrix, cada fila corresponde a un token, típicamente un palabra ó un caracter. Es decir cada fila es un vector que representa una palabra.

- Estos vectores (típicamente) son **word embeddings** (**word2vec** ó **GloVe**) ó **one-hot vectors.**

CNN para Secuencias, Conv1D

Padding = 0

tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3

t,d,r	−1.0
d,r,t	−0.5
r,t,k	−3.6
t,k,g	−0.2
k,g,o	0.3

Apply a **filter** (or **kernel**) of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

Creditos: Christopher Manning

CNN para Secuencias, Conv1D

Padding = 1

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Apply a **filter** (or **kernel**) of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

Ø,t,d	−0.6
t,d,r	−1.0
d,r,t	−0.5
r,t,k	−3.6
t,k,g	−0.2
k,g,o	0.3
g,o,Ø	−0.5

https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/may_2026/Clase%205/C%C3%B3digo/Ejercicio_resuelto%2C_CNN_para_Secuencias%2C_Conv1D.ipynb

Creditos: Christopher Manning

CNN para Secuencias, Conv1D

Padding = 0, 3 Kernels

ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
ø	0.0	0.0	0.0	0.0

ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,ø	−0.5	−0.9	0.1

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Creditos: Christopher Manning

CNN para Secuencias, Conv1D

Padding = 0, 3 Kernels, Max Pooling

Ø	0.0	0.0	0.0	0.0
tentative	0.2	0.1	−0.3	0.4
deal	0.5	0.2	−0.3	−0.1
reached	−0.1	−0.3	−0.2	0.4
to	0.3	−0.3	0.1	0.1
keep	0.2	−0.3	0.4	0.2
government	0.1	0.2	−0.1	−0.1
open	−0.4	−0.4	0.2	0.3
Ø	0.0	0.0	0.0	0.0

Ø,t,d	−0.6	0.2	1.4
t,d,r	−1.0	1.6	−1.0
d,r,t	−0.5	−0.1	0.8
r,t,k	−3.6	0.3	0.3
t,k,g	−0.2	0.1	1.2
k,g,o	0.3	0.6	0.9
g,o,Ø	−0.5	−0.9	0.1
max p	0.3	1.6	1.4

Apply 3 **filters** of size 3

3	1	2	−3
−1	2	1	−3
1	1	−1	1

1	0	0	1
1	0	−1	−1
0	1	0	1

1	−1	2	−1
1	0	−1	3
0	2	2	1

Creditos: Christopher Manning

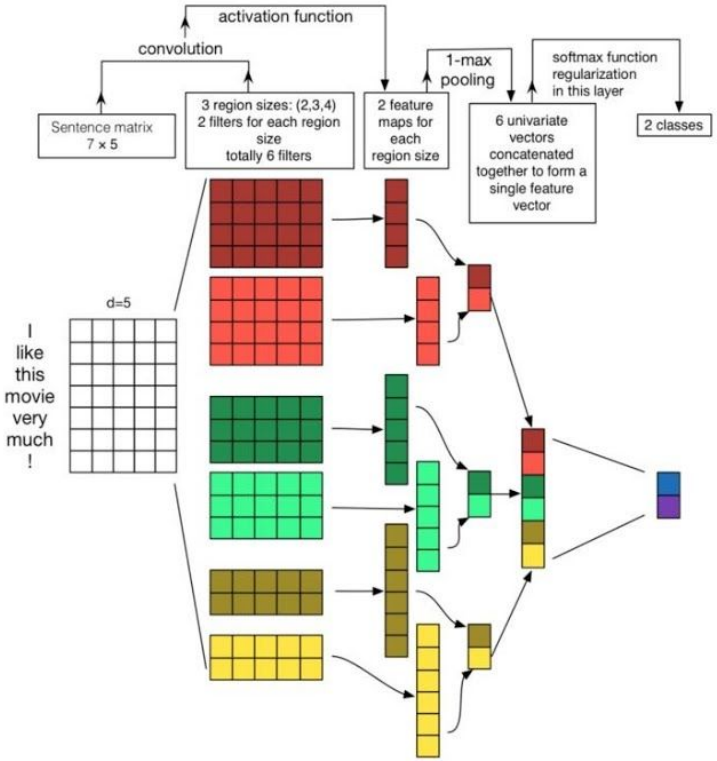
CNN para Secuencias, Arquitectura

From:

Zhang and Wallace (2015) A Sensitivity Analysis of (and Practitioners' Guide to) Convolutional Neural Networks for Sentence Classification

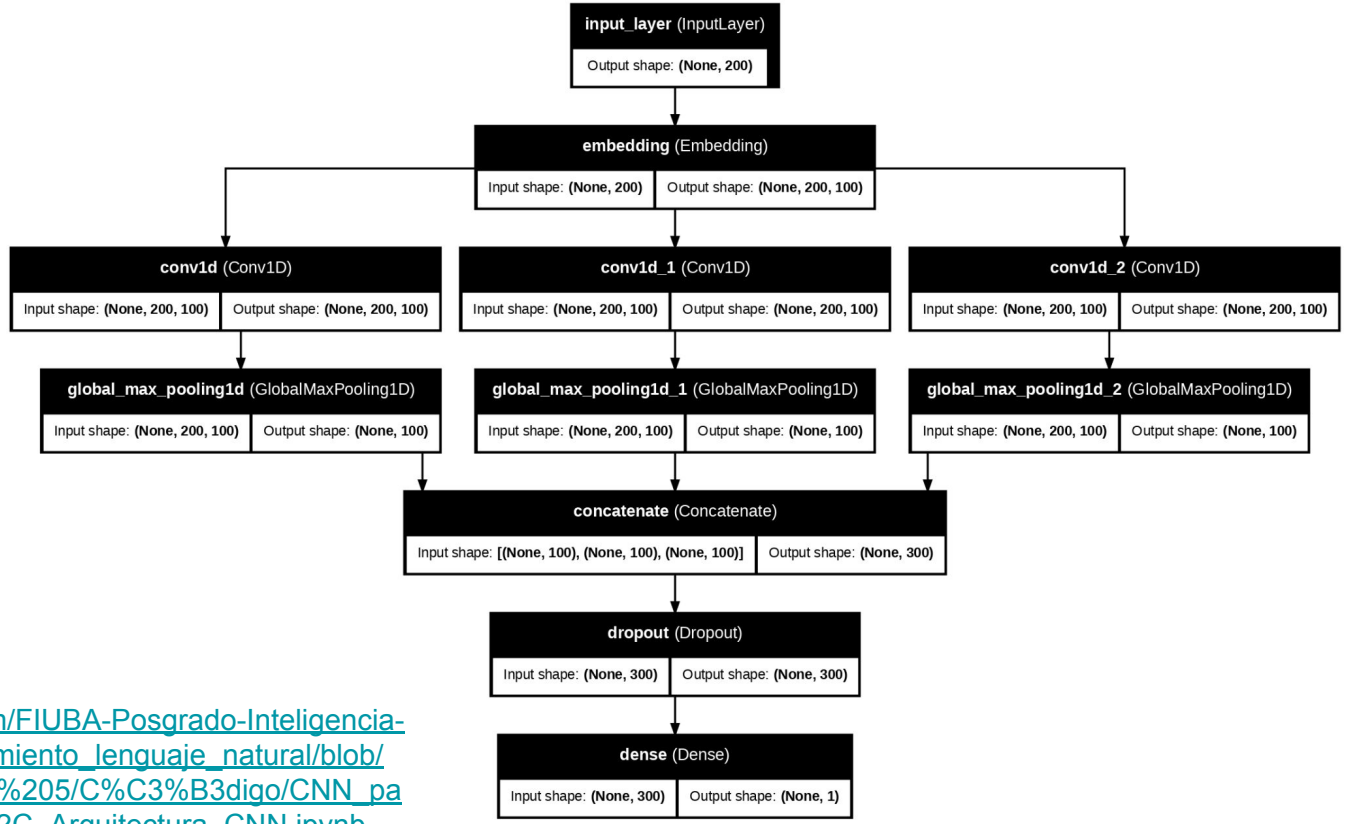
https://arxiv.org/pdf/1510.03820.pdf

(follow on paper, not famous, but a nice picture)



Creditos: Christopher Manning

CNN para Secuencias, Arquitectura



https://github.com/FIUBA-Posgrado-Inteligencia-Artificial/procesamiento_lenguaje_natural/blob/may_2026/Clase%205/C%C3%B3digo/CNN_para_Secuencias%2C_Arquitectura_CNN.ipynb

Creditos: Christopher Manning

CNN para Secuencias

CNN funciona pero no es lo más óptimo para NLP:

- **Contexto largo pobre:** solo capturan relaciones locales.
- **Sin atención global:** no modelan bien relaciones entre palabras lejanas.

Una CNN procesa en ventanas pequeñas, por lo que pierde esa relación a larga distancia. **Necesitamos otro mecanismo.**

Mecanismo de atención

En general, un mecanismo de atención es una transformación de secuencias de embeddings a secuencias de embeddings de forma ponderada y paralela.

Existen varios tipos pero las operaciones fundamentales consisten en:

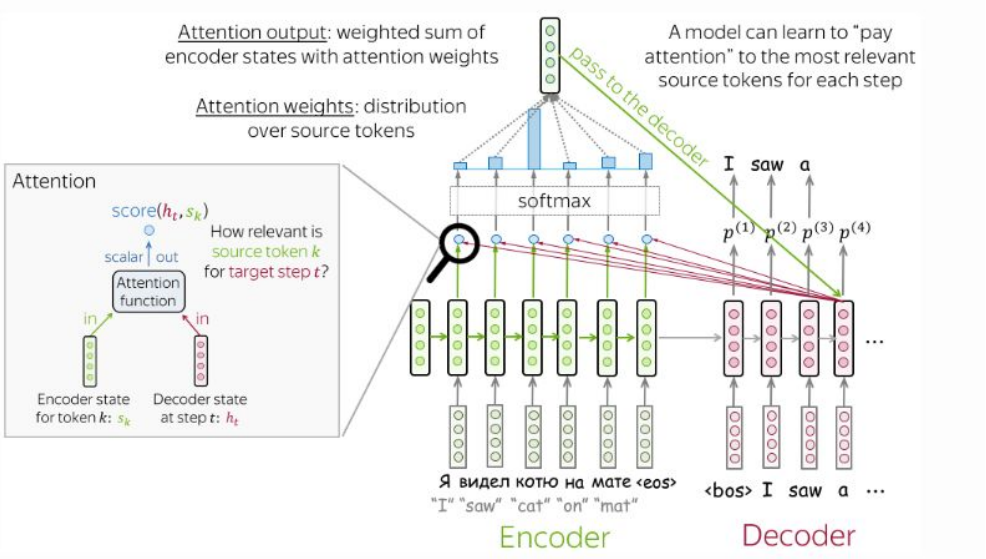
- *El cálculo un vector de pesos/scores de atención .*
- *La construcción de un vector ponderado que es fácilmente paralelizable a lo largo del tamaño de la secuencia.*

¡La degradación del gradiente es independiente del tamaño de la secuencia! Pero se debe fijar el tamaño máximo de secuencia a procesar.

*A futuro veremos que el mecanismo de **self-attention** es el que utilizan las arquitecturas **transformer**.*

Mecanismos de atención

Atención para Encoding-decoding



Self-attention (transformers)

Each vector receives three representations ("roles")

W_Q × = Query: vector **from** which the attention is looking

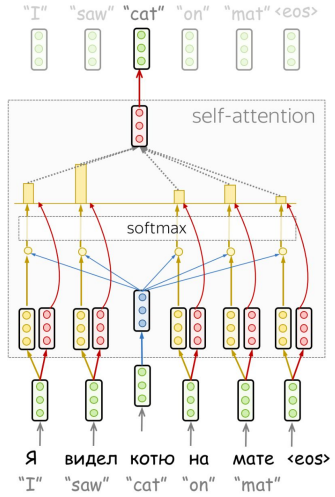
"Hey there, do you have this information?"

W_K × = Key: vector **at** which the query looks to compute weights

"Hi, I have this information – give me a large weight!"

W_V × = Value: their weighted sum is attention output

"Here's the information I have!"



https://lena-voita.github.io/resources/lectures/seq2seq/transformer/encoder_self_attention.mp4